

TradeVoice: technical documentation

NAIC 2026, Innovation and Enterprise track, PS2 Voice-First Access.

- **Live build:** <https://tradevoice.duckdns.org> (the web app is at `/app`).
- **Code:** <https://github.com/Godzilla-lab/tradevoice>
- **This document:** architecture, setup and usage, so that someone outside the team can run TradeVoice or build on it. It describes the code as it is on 9 October 2026.

1. What it is

TradeVoice keeps the books for Nigerian market traders who would rather talk than type. A trader says, for example, "Iya Bisi took 2 bags of rice for 60,000, she will pay Friday" in English, Pidgin, Yorùbá, Hausa or Igbo. TradeVoice turns that into a record (who, what, how much, credit or paid, pay-by date), shows it on a check card, and saves it only when the trader says yes. From the book it answers questions ("Who owes me the most?"), drafts reminders with pay links, reads photos of notebook pages, and makes a report the trader can share with a lender.

Three rules shape the code:

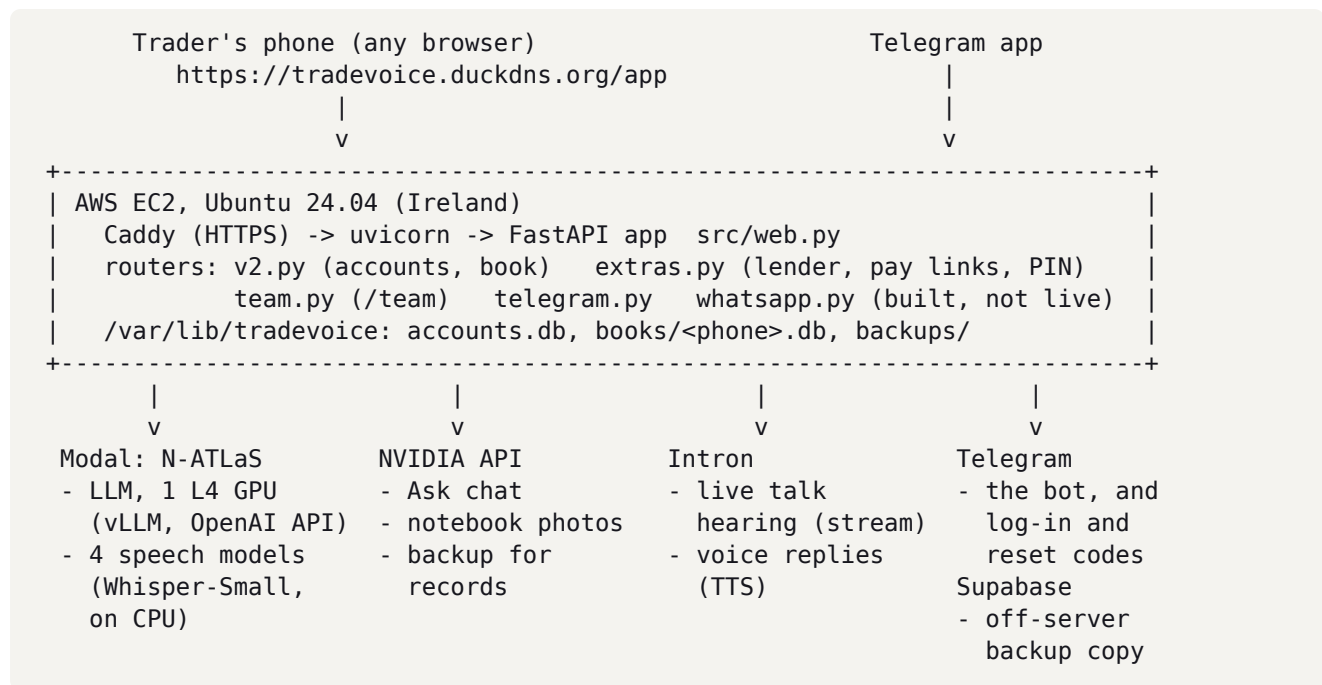
1. **Nothing is saved without the trader's yes.** Every record is read back first.
2. **AI reads and phrases; code does the maths.** Totals, balances and every amount are worked out in Python from the book. An amount the trader did not say is never kept.
3. **One book per phone number.** Each trader's records live in their own SQLite file.

The main language model and the speech models are N-ATLaS. N-ATLaS is an initiative of the Federal Ministry of Communications, Innovation and Digital Economy, and powered by Awarri Technologies.

Ways in:

Channel	Status	Code
Web app (phone browser, installable)	Live, the main channel	<code>src/web.py</code> , <code>web/app.html</code> , <code>web/live.js</code>
Telegram bot	Live, free, optional	<code>src/telegram.py</code>
WhatsApp bot	Built and tested, not live: the team could not get WhatsApp Business API access. It switches on when the Meta keys are set	<code>src/whatsapp.py</code>

2. Architecture



Who does each job

Job	Main	When the main one fails or is late
Hear a voice note	N-ATLaS speech model for the trader's language, on Modal (<code>src/asr.py</code>)	The trader is asked to send it again or type it. Intron and Spitch hearing exist only for comparison tests
Hear live talk (spoken back and forth)	Intron streaming speech-to-text, relayed by our server (<code>src/intron_live.py</code>)	N-ATLaS speech models, one turn at a time (<code>/api/hear</code>)
Merge two hearings of one note (Yorùbá, Hausa, Igbo)	N-ATLaS LLM (<code>src/hearing.py</code>)	The trader's language model's words are used
Write a record from words	N-ATLaS LLM with guided JSON (<code>src/extract.py</code> , <code>src/llm.py</code>)	NVIDIA API models (<code>LLM_MODELS</code>), then the offline rules
Answer in the Ask chat	English and Pidgin: NVIDIA models first, N-ATLaS next. Yorùbá, Hausa, Igbo: N-ATLaS first (<code>src/agent.py</code>)	The other models, then the rules (<code>src/converse.py</code>)
Read a photo of a notebook page	NVIDIA vision models (<code>src/vision.py</code> , <code>src/photo.py</code>)	"I can't read photos right now"
Speak replies	Intron text-to-speech (<code>src/tts.py</code>)	The reply stays as text
Prove a phone number	Live setup: no code at sign-up (number and password). Log-in and reset codes go to Telegram for numbers shared with the bot (<code>src/v2.py</code> , <code>src/telegram.py</code>)	The team resets a password by hand. See section 7

How a request reaches the right book

Every HTTP request passes through the `BookPerTrader` middleware in `src/web.py`:

1. It reads the session cookie `tv_auth` and finds the phone number (`accounts.phone_for`).
2. Not logged in: every `/api/` path answers 401 `{"login": true}`, except `/api/auth/...`, `/api/ui`, `/api/status` and `/api/voice_check`.
3. If the trader set a PIN, the request needs a valid `X-TV-Unlock` header, else 423 `{"locked": true}`.
4. `ledger.use_book(phone)` points this request (a Python context variable) at `books/<phone>.db`. Every module then reads and writes that trader's book only.

The live hearing WebSocket checks the same cookie and closes with code 4401 when logged out.

Front end

File	What it is
<code>design/tradevoice-2.0/app.html</code>	The designer's app. Never edited by hand in the build
<code>scripts/build_app.py</code>	Builds <code>web/app.html</code> from the design: swaps a short list of exact snippets (PATCHES) so accounts live on the server, hides test tools unless <code>?demo=1</code> , loads <code>web/live.js</code> . Stops and names any snippet the designer changed. <code>eval/test_design.py</code> checks that the CSS and page structure stay identical
<code>web/live.js</code>	Runtime behaviour on top of the design: the real book, voice notes, live talk, reminders, photo scan, lender link, the Ask chat
<code>web/landing.html</code> , <code>web/channels.js</code>	The website at <code>/</code> (built from <code>design/tradevoice-2.0/landing2.html</code>); <code>channels.js</code> hides or rewords what can't work today
<code>web/privacy.html</code>	The privacy notice at <code>/privacy</code>
<code>web/team.html</code>	The team dashboard at <code>/team</code> (needs <code>ADMIN_TOKEN</code>)
<code>web/sw.js</code> , <code>web/manifest.webmanifest</code>	Offline start and "Add to Home Screen"

When the server sends `/app` or `/`, it adds `window.TV_CH` to the page (`web._page`, from `v2.channels()`), so the page knows which ways in work today: `codes` ("whatsapp", "sms" or empty), `wa`, `sms`, `tg` (the Telegram bot's name), `nocode` and `team` (the privacy contact).

Repository map

Path	What it does
<code>src/web.py</code>	FastAPI app: middleware, talk, voice, photo, Ask chat, customers, screens, pages
<code>src/v2.py</code>	Accounts (sign-up, log in, reset, delete) and the book shaped for the app's screens

Path	What it does
<code>src/accounts.py</code>	Phone numbers, one-time codes, sessions (all hashed)
<code>src/ledger.py</code>	The SQLite book: entries, customers, messages, reminders, balances, credit profile
<code>src/converse.py</code>	One conversation over one book: routes each message (record, yes/no, question, list, reminder)
<code>src/extract.py</code>	Words to a record: offline rules, N-ATLaS with <code>REC_SCHEMA</code> and <code>SHOTS</code> , guards
<code>src/llm.py</code>	Every language-model call, with N-ATLaS first and the backups in order
<code>src/agent.py</code> , <code>src/tools.py</code> , <code>src/askbook.py</code>	The Ask chat: the model picks a tool, code runs it on the book, code checks every number
<code>src/asr.py</code> , <code>src/hearing.py</code>	Hearing voice notes with N-ATLaS; merging two hearings; what to ask again
<code>src/intron_live.py</code> , <code>src/tts.py</code>	Live talk with Intron streaming; Intron voice replies
<code>src/vision.py</code> , <code>src/photo.py</code>	Notebook photos to draft rows
<code>src/natlas_watch.py</code>	Keeps N-ATLaS warm in market hours; alerts the team
<code>src/telegram.py</code> , <code>src/whatsapp.py</code> , <code>src/sms.py</code>	The Telegram bot; the WhatsApp bot; optional SMS codes (off unless its keys are set)
<code>src/extras.py</code>	Lender link, pay links (Paystack), automatic reminders, receipts, PIN, CSV export
<code>src/events.py</code> , <code>src/team.py</code> , <code>src/training.py</code>	Anonymous usage log; team dashboard; opt-in data for improving TradeVoice
<code>src/insights.py</code> , <code>src/readaloud.py</code> , <code>src/assistant.py</code> , <code>src/tax.py</code> , <code>src/ui_text.py</code>	Forecast and statements; screens read aloud; screen help; tax facts; screen words in 4 languages
<code>deploy/modal_natlas.py</code> , <code>deploy/modal_asr.py</code>	N-ATLaS LLM and speech models on Modal
<code>deploy/server/</code>	Server setup, keys, updates, checks, backups (section 6)
<code>deploy/modal_web.py</code>	An earlier way to host the web app on Modal. Not used by the live server
<code>scripts/</code>	Backup, pilot check, accounts, reports, model checks, front-end build
<code>eval/</code>	Automated tests and accuracy benchmarks (section 11)

3. How a voice note becomes a record

This is the Talk button in the web app. File and function names are given for each step.

Step	Where	What happens
1	<code>web/live.js</code> (Talk card)	The trader taps the mic. The page calls <code>POST /api/warm (natlas_watch.wake)</code> , which pings both Modal apps if they may be asleep, at most once a minute. The page records until about 1.2 s of quiet, a tap, or 30 s
2	<code>POST /api/voice</code> , <code>BookPerTrader</code>	The upload carries the language and <code>consent=yes</code> . The middleware opens this trader's book
3	<code>web._hear</code> , <code>asr.transcribe_auto</code> , <code>asr._natlas_transcribe</code>	The audio goes to <code>NATLAS_ASR_URL/transcribe</code> with the language, the trader's own customer names and items as hints, <code>prep=1</code> , and <code>also=english</code> for Yorùbá, Hausa and Igbo. If the server is asleep: one try with 60 s, then a wake call to <code>/health</code> and one more try with 240 s
4	<code>deploy/modal_asr.py</code> <code>ASR.web</code>	ffmpeg makes 16 kHz mono audio. A silence check returns <code>no_speech</code> . Light prep trims quiet ends and levels the volume (no denoising). The Whisper-Small model for the language runs, with the English model in parallel when asked. Words with a token probability under 0.4 come back as "unsure"
5	<code>asr.looping</code> , <code>hearing.merge</code> , <code>hearing.check</code>	A transcript that repeats itself is rejected. For two hearings, N-ATLaS writes the one sentence the trader most likely said (8 s limit). The merge is dropped if it holds an amount neither model heard or is much longer than both. Amounts the two models heard differently, and unsure words, are kept so the chat asks again for just that part
6	<code>asr.transcribe_auto</code>	If the words look like another of the local languages, the note is heard again in that language
7	<code>web._hear</code>	<code>training.keep</code> stores the audio only for a trader who said yes to helping improve TradeVoice. Then the file is deleted
8	<code>web._say</code> , <code>converse.reply</code>	Runs inside <code>llm.budget(ASK_AI_SECONDS)</code> (20 s), so the trader gets an answer in time even while N-ATLaS wakes up. The reply follows the language of the message. The message is routed: yes/no, question, list, promise to pay, or a new record
9	<code>converse._record</code> , <code>extract.extract</code>	<code>extract.rule_extract</code> reads the record offline first. Then <code>extract.llm_extract</code> calls <code>llm.chat(..., shots=SHOTS, schema=REC_SCHEMA)</code> . N-ATLaS is tried first, with guided JSON and the model card's settings (temperature 0.1, repetition penalty 1.12, today's date in the chat template). If it fails or takes longer than <code>NATLAS_TIMEOUT</code> (20 s), the NVIDIA models in <code>LLM_MODELS</code> are tried. If every model fails, the rules' record is used

Step	Where	What happens
10	<code>extract._normalise</code> , <code>_sell_guard</code> , <code>_check_guard</code>	Code fills empty fields from the rules, multiplies "each" prices by the quantity, and corrects the type or amount when the words are clear (for example "not paid yet" is credit; an amount that was never said is replaced)
11	<code>converse._pick_customer</code> , <code>_ask_again</code> , <code>price_check</code> , <code>draft_limit</code>	The name is matched to the book ("Which Alhaji?"). A missing amount gets "How much?". An unclear amount is asked again. A price far from this trader's usual price, or a sale over a customer's credit limit, is flagged
12	<code>web._reply_json</code> , <code>web._draft</code>	The page gets the check card: type, amount, customer, item, quantity, due date, and the fields TradeVoice is unsure of (shown with "?"). Nothing is saved yet. The spoken read-back is made at once in the background (<code>tts.speak</code> , <code>Intron</code>) and fetched from <code>/api/speak/{id}</code>
13	Save: <code>POST /api/message</code> with "yes", <code>converse._confirm</code> , <code>ledger.add_entry</code>	The record goes into the trader's book and the reply gives the new balance. "Change" uses <code>/api/draft</code> . "Undo" within 2 minutes uses <code>/api/v2/undo_last</code>

Other ways in follow the same path from step 8:

- **Typed words:** `POST /api/message` goes straight to `converse.reply`.
- **Live talk:** the page streams the microphone to the WebSocket `/api/live/hear`. The server relays it to Intron's streaming speech-to-text and sends the words back as they come; the Intron key never leaves the server. When the trader stops, `POST /api/say` gets the reply, and its voice comes in short pieces from `/api/speak/{id}/{n}`. If Intron is off, each turn is heard by N-ATLaS through `POST /api/hear`. In live talk a record the rules read completely is answered at once (`LIVE_FAST_RECORDS`), and the two hearings are compared by code but not merged.
- **Ask chat:** `POST /api/ask` uses `agent.answer`: the model chooses a tool (`book_summary`, `who_owes`, `i_owe`, `customers`, `sales`, `calculate`, `date`, `record`), code runs it on the book, and code checks that every number in the reply came from the book, a tool or the trader's words. A new record said in the chat goes to the record flow above and waits for yes.
- **Notebook photo:** `POST /api/photo` or `POST /api/ask/photo` calls `photo.read`: an NVIDIA vision model copies each money line, `extract.extract_many` turns the lines into draft rows, the trader ticks and fixes them, and `POST /api/save_rows` saves the ticked rows. The photo is deleted after reading.
- **Telegram and WhatsApp:** `whatsapp.handle` uses the same hearing (`asr.transcribe_auto`), the same brain (`converse.reply`) and the same photo reader, on the same book for the same number.

Work that takes longer than 25 s (a sleeping N-ATLaS, a photo, a long list) carries on in the background. The page gets `{"job": id}` and collects the answer from `/api/job/{id}`, because phone browsers give up on a request after about 60 s.

4. Run it on your own machine

These steps are for Linux or macOS. The server itself runs Python 3.12 on Ubuntu 24.04.

You need: Python 3.10+, git, and ffmpeg (brew install ffmpeg or sudo apt-get install ffmpeg).

```
git clone https://github.com/Godzilla-lab/tradevoice.git
cd tradevoice
python3 -m venv .env
source .env/bin/activate
pip install -r requirements.txt
cp .env.example .env          # then edit .env: see below
python src/web.py             # http://localhost:8000 (another port: PORT=8001)
```

Open <http://localhost:8000/app> for the web app and <http://localhost:8000/> for the website. Browsers allow the microphone on `localhost`; a phone needs an HTTPS address (section 6).

`requirements.txt` installs `requirements-server.txt` (exactly what the live server runs, with pinned versions) plus optional extras: `faster-whisper` for hearing on your own computer, `spitch` for the speech comparison tests, and `markdown` for making the NAIC PDFs. The server installs only `requirements-server.txt`. There are no separate admin screens: the old Gradio screens (`src/app.py`, `src/note.py`, `/admin`) were removed, and the team uses `/team`.

Edit `.env` before you start. It is read automatically at start (`src/settings.py`) and never committed.

- The line `NVIDIA_API_KEY=nvapi-your-key-here` is a placeholder. Put in your own key from build.nvidia.com, or turn the line into a comment if you have none.
- For N-ATLaS, set `NATLAS_URL`, `NATLAS_ASR_URL` and `NATLAS_KEY` after section 5.

What works with which keys:

You set	What you get
Nothing	Accounts, the book, customers, typed records and questions through the offline rules, reminders, statements. No photos. No voice replies
<code>NATLAS_URL</code> , <code>NATLAS_KEY</code>	N-ATLaS writes the records and answers in Yorùbá, Hausa and Igbo
<code>NATLAS_ASR_URL</code>	Voice notes in all five languages, heard by N-ATLaS
<code>NVIDIA_API_KEY</code>	Notebook photos, the Ask chat's main models, backups for records
<code>INTRON_API_KEY</code>	Voice replies and live talk

Without `NATLAS_ASR_URL`, voice notes are heard by Intron if `INTRON_API_KEY` is set, otherwise by Whisper on your own computer (`faster-whisper`, installed by `requirements.txt`), which covers English and Pidgin only.

Signing up on your machine. As on the live server, sign-up is phone number and password, with no code. Use made-up numbers. To see the code screens, set `AUTH_DEMO=1`: the code then shows on screen as a pretend WhatsApp message. `AUTH_DEMO` is for tests on your own computer only. The live server ignores it (`TV_PUBLIC=1`).

A first record without the browser (made-up number and names):

```

curl -c jar -H 'Content-Type: application/json' \
  -d '{"phone":"0803 000 0417","purpose":"signup"}' \
  localhost:8000/api/auth/v2/code/start
# -> {"login_id": "...", "nocode": true, ...}
# the app hashes the password before sending it
PW=$(printf 'tv:pw:%s' 'my-test-password' | shasum -a 256 | cut -d' ' -f1)
curl -b jar -c jar -H 'Content-Type: application/json' \
  -d '{"login_id":"LOGIN_ID","pw":"'PW',"name":"Iya Bisi","biz":"Iya Bisi Shop"}' \
  localhost:8000/api/auth/v2/signup
curl -b jar -H 'Content-Type: application/json' \
  -d '{"session":"s1","text":"Mama Ngozi took 2 bags rice 60k, she will pay Friday"}' \
  localhost:8000/api/message # -> a draft card, nothing saved
curl -b jar -H 'Content-Type: application/json' -d '{"session":"s1","text":"yes"}' \
  localhost:8000/api/message # -> "Saved."
curl -b jar localhost:8000/api/v2/book

```

Replace `LOGIN_ID` with the value from the first answer. On Linux, `sha256sum` does the same as `shasum -a 256`. Local data goes to `accounts.db`, `tradevoice.db` and `books/` in the folder you started from (all ignored by git).

5. Deploy N-ATLaS on Modal

Two Modal apps serve the N-ATLaS models. Both are in `deploy/` and both read one Modal secret named `natlas`.

App	File	Models	Hardware
tradevoice-natlas	deploy/modal_natlas.py	NCAIR1/N-ATLaS (Llama-3 8B), served by vLLM 0.10.2 as an OpenAI-compatible API, name <code>natlas</code> , <code>bfloat16</code> , <code>--max-model-len 8192</code>	1 NVIDIA L4 GPU (24 GB)
tradevoice-natlas-asr	deploy/modal_asr.py	NCAIR1/Yoruba-ASR, NCAIR1/Hausa-ASR, NCAIR1/Igbo-ASR, NCAIR1/NigerianAccentedEnglish (all Whisper-Small fine-tunes; English and Pidgin use NigerianAccentedEnglish)	CPU, 8 cores, 8 GB memory

The links they get (WORKSPACE is your Modal workspace name), and the settings they go into:

```

NATLAS_URL=https://WORKSPACE--tradevoice-natlas-serve.modal.run/v1
NATLAS_ASR_URL=https://WORKSPACE--tradevoice-natlas-asr-asr-web.modal.run

```

The official weights are served, not a quantised copy. Library versions are pinned in both files.

Steps. Do these on your own computer, and never paste keys into a chat or a commit.

Step 1. On Hugging Face, open each model page above and accept any conditions it asks for (Yoruba-ASR asks). Make a read token for that account.

Step 2. Install Modal, log in, and make the secret. Choose a long random string for `NATLAS_KEY`; the TradeVoice server sends it to both apps as a Bearer token.

```

pip install modal && modal setup
modal secret create natlas NATLAS_KEY=YOUR_LONG_RANDOM_STRING \
  HF_TOKEN=YOUR_HF_READ_TOKEN

```

Step 3. Deploy both apps. Each prints its link.

```

modal deploy deploy/modal_natlas.py
modal deploy deploy/modal_asr.py

```

Step 4. Put the two links and the same `NATLAS_KEY` in `.env` (on the server: `keys.sh`, section 6). The LLM link needs `/v1` at the end.

Step 5. Check them. The first call after a deploy downloads the weights and can take several minutes.

```

curl -H "Authorization: Bearer $NATLAS_KEY" "$NATLAS_URL/models"
curl -H "Authorization: Bearer $NATLAS_KEY" "$NATLAS_ASR_URL/health"
curl -H "Authorization: Bearer $NATLAS_KEY" -F file=@note.ogg -F lang=yoruba \
  "$NATLAS_ASR_URL/transcribe"
python scripts/check_models.py --natlas # wakes N-ATLaS, times one record a language

```

The speech server's API. POST `/transcribe` (form fields): `file` (any audio format), `lang` (yoruba, hausa, igbo, english; pidgin means english), `prompt` (optional hint words), `prep` (1 or 0), also (a second language model to run on the same note). Answer: `text`, `model`, `seconds`, `ms`, `unsure` (words with their probability), `confidence`, `no_speech` when there was only silence, and `also` when asked. GET `/health` answers when the four models are loaded. Both need the `Authorization: Bearer NATLAS_KEY` header.

Sleeping and waking. Both apps scale to zero after 60 idle minutes. A sleeping LLM takes about 3 to 4 minutes to start, the speech server a minute or two. Meanwhile the backups answer (NVIDIA models, then the rules), so traders still get a reply. During market hours the web server keeps both awake:

`src/natlas_watch.py` pings them every 10 minutes from 7am to 8pm Lagos time when `NATLAS_WATCH=1`, and tells the team (Telegram, WhatsApp) after two failed checks in a row. The L4 bills only while a container runs; keeping it warm in market hours costs about \$10 a day.

Deploy-time options (set in the shell that runs `modal deploy`, not in `.env`):

Option	Default	What it does
<code>NATLAS_WARM</code>	0	1 keeps one GPU running day and night
<code>NATLAS_SLEEP_MIN</code>	60	Idle minutes before a container sleeps (both apps)
<code>NATLAS_GPU</code>	L4	GPU type for the LLM
<code>NATLAS_QUANT</code>	(none)	<code>fp8</code> serves 8-bit weights for faster answers; check accuracy first
<code>NATLAS_MODEL_ID</code>	NCAIR1/N-ATLaS	Another model id deploys a separate app, <code>tradevoice-natlas-base</code> , for benchmarks against the base model
<code>NATLAS_ASR_GPU</code>	(none, CPU)	For example <code>T4</code> to run the speech models on a GPU
<code>NATLAS_ASR_CPU</code>	8	CPU cores for the speech server
<code>NATLAS_ASR_UNSURE</code>	0.4	Below this token probability a word is marked unsure

6. Deploy the server

The live server is one small Ubuntu machine. Everything is done by the scripts in `deploy/server/`, run with `sudo`.

1. The machine (AWS). EC2, region Europe (Ireland) `eu-west-1`, image Ubuntu Server 24.04 LTS (64-bit x86), type `t3.micro`, 20 GiB gp3 disk. Security group: SSH from your own IP; HTTP and HTTPS from anywhere (HTTP is needed for the certificate). Any cloud with Ubuntu 24.04 works the same way.

2. A free address. Make a name at `duckdns.org` (for example `tradevoice`) and copy the account's token.

3. Install. Log in with SSH, then:

```
sudo git clone https://github.com/Godzilla-lab/tradevoice /opt/tradevoice/app
sudo bash /opt/tradevoice/app/deploy/server/setup.sh
```

`setup.sh` asks for the DuckDNS name and token once (the token is checked with DuckDNS and never shown), then:

- installs Python, ffmpeg, SQLite, Caddy and automatic security updates; adds 2 GB swap on a 1 GB machine;
- makes a system user `tradevoice`, the virtual environment `/opt/tradevoice/venv` (from `requirements-server.txt`) and the data folder `/var/lib/tradevoice` (books, backups), readable by that user only;
- makes `.env` from `.env.example` once, sets `PUBLIC_URL`, and turns laptop-only settings into comments;
- writes `/etc/tradevoice/paths.env` (data paths, `PORT=8000`, `TV_PUBLIC=1`);
- installs systemd units: `tradevoice` (one unicorn process on `127.0.0.1:8000`, restarts by itself), `tradevoice-backup.timer` (every hour), `duckdns.timer` (keeps the name pointing at the machine, every 5 minutes);
- writes the Caddy config: HTTPS certificate by itself for `NAME.duckdns.org`, uploads up to 30 MB, proxy to the app;
- opens ports 80 and 443, starts everything, checks `/api/status` locally and over HTTPS, and makes a first backup.

It is safe to run again. `--duckdns` asks for the name and token again.

4. Keys. `.env` lives at `/opt/tradevoice/app/.env` (root and the app's user only). Keys go in only through:

```
sudo bash /opt/tradevoice/app/deploy/server/keys.sh          # every key
sudo bash /opt/tradevoice/app/deploy/server/keys.sh NATLAS_KEY # just one
```

It asks for each key in order of importance, hides secret input, checks the shape (for example links start with `https://`), keeps a key on Enter and removes it on `-`. Then it restarts the app and shows what it now uses (brain, hearing, WhatsApp, voice, keep-awake), never the keys. If you leave `ADMIN_TOKEN` empty it makes one for you.

5. Webhooks. Telegram: nothing to do; the app registers `PUBLIC_URL/telegram/webhook` at start.

Paystack: in the dashboard set the webhook to `https://NAME.duckdns.org/paystack/webhook`.

WhatsApp (when access exists): set the callback `https://NAME.duckdns.org/whatsapp/webhook` and the same `WHATSAPP_VERIFY_TOKEN` in Meta's settings.

6. Check before traders use it.

```
sudo bash /opt/tradevoice/app/deploy/server/preflight.sh
sudo bash /opt/tradevoice/app/deploy/server/preflight.sh --only "Backups,Disk space"
```

One line per item, PASS, WARN or FAIL: app answering, public link, N-ATLaS brain, N-ATLaS hearing, NVIDIA models, Intron voice replies, Intron live hearing, WhatsApp bot, sign-up codes, Telegram bot, settings, keep N-ATLaS awake, backups, disk space, tools and folders. It sends the smallest possible request to each service and never prints a key, so the output can be shared with anyone helping.

7. Updates.

```
sudo bash /opt/tradevoice/app/deploy/server/update.sh # branch main
sudo BRANCH=other-branch bash /opt/tradevoice/app/deploy/server/update.sh
```

It refuses if files on the server were edited by hand, makes a backup, checks out the new commit, installs `requirements-server.txt`, restarts and waits for `/api/status`. If the new code does not start, it goes back to the commit that worked. It tells you when `setup.sh` changed and should be run again.

Other server commands (all `sudo bash /opt/tradevoice/app/deploy/server/NAME.sh`):

Script	What it does
<code>restore.sh FILE_OR_FOLDER</code>	Puts books and accounts back from a backup file or a folder. Stops the app for a few seconds; keeps the replaced files in <code>backups/before-restore-*</code>
<code>add_account.sh PHONE "NAME" "BUSINESS"</code>	Makes a tester's account and prints a temporary password. <code>add_account.sh PHONE --reset</code> resets a forgotten password (new temporary password, every phone logged out)
<code>erase_account.sh PHONE [--yes]</code>	Erases an account now, when its owner asks sooner than 90 days. Without <code>--yes</code> it only shows what would go
<code>validation_report.sh [--start DATE --end DATE]</code>	The NAIC validation page from the usage log (counts only, team phones left out), saved in your home folder
<code>speed.sh [--days N]</code>	How long each step of a voice turn takes, from the usage log
<code>speech_ab.sh FOLDER</code>	Speech fixes on and off, on consented test voice notes
<code>tools_eval.sh [--show]</code>	Tool-choice accuracy per language with the live N-ATLaS, on a throwaway book
<code>whatsapp_templates.sh --print/--status/--create</code>	The four WhatsApp message templates (for later)
<code>store_waitlist.sh</code>	Who asked to hear when the app is in the app stores

Logs: `sudo journalctl -u tradevoice -f`. **Backups:** `sudo ls -lh /var/lib/tradevoice/backups`.

Files on the server:

Path	What it holds
/opt/tradevoice/app	The code (a git checkout, owned by root) and .env
/opt/tradevoice/venv	Python packages
/etc/tradevoice/paths.env	ACCOUNTS_DB, BOOKS_DIR, DB_PATH, BACKUP_DIR, PORT, TV_PUBLIC=1
/etc/tradevoice/duckdns.env	The DuckDNS name and token (root only)
/var/lib/tradevoice/accounts.db	Accounts, sessions, usage log, links (section 10)
/var/lib/tradevoice/books/	One SQLite book per phone number
/var/lib/tradevoice/backups/	Hourly backups
/var/lib/tradevoice/training/	Only for traders who said yes to helping improve TradeVoice

Only one app process runs, on purpose: every book is a SQLite file, and one writer at a time is safe.

7. Sign-up, codes and channels

Accounts

A trader signs up with their phone number, a password, their name and business name, and then logs in with phone and password. The browser sends `sha256("tv:pw:" + password)`; the server stores only PBKDF2 of that (200,000 rounds, random salt). 5 wrong passwords mean a 30-second wait.

The live setup: no code at sign-up

The live server has no WhatsApp and no SMS sender, so **sign-up is phone number and password, with no code** (`v2.channels()` reports `nocode`). Only a number with no account can sign up this way, so nobody can take over an existing book: opening a book always needs its password.

- **Telegram users can confirm their number** with **Confirm on Telegram**. The button is on the code screen (log in with a code, forgotten password, a new phone number; at sign-up only when `SIGNUP_CODE=required`). It opens the bot with `/start login-WORD`; the trader taps **Share my phone number**, Telegram sends the number on their account, and the page carries on by itself (`/api/auth/v2/code/poll`). In Me, **Connect my Telegram** links the bot to the same book.
- **Forgotten password:** for a number shared with the Telegram bot, the reset code is sent in Telegram (or the trader uses Confirm on Telegram). Otherwise the page asks the trader to contact the team, and the team resets it on the server: `sudo bash /opt/tradevoice/app/deploy/server/add_account.sh PHONE --reset` prints a temporary password and logs out every phone. The trader changes it in Me, Password.
- **Changing the phone number** needs the new number confirmed, which on the live setup means Confirm on Telegram.

How `POST /api/auth/v2/code/start` decides

`src/v2.py code_start` handles sign-up, log in with a code, reset and a new phone number:

Order	Way	When
1	WhatsApp message, or "Send it on WhatsApp" (<code>LOGIN_WORD</code> sent to the bot)	Only when the WhatsApp keys are set (not live)
2	A code in Telegram	Log in and reset, for a number already shared with the Telegram bot
3	SMS	Only when <code>TEXTBEE_API_KEY</code> and <code>TEXTBEE_DEVICE_ID</code> are set. <code>src/sms.py</code> is optional plumbing: off on the live server, not asked for by <code>keys.sh</code> , still covered by <code>eval/test_sms.py</code>
4	No code	Sign-up, when nothing above can send and <code>SIGNUP_CODE</code> is not required
5	Confirm on Telegram	Whenever the Telegram bot is set up and no code was sent (log in, reset, new phone number; also sign-up when <code>SIGNUP_CODE=required</code>)

Limits: 5 code requests an hour per number and 20 per connection; a code lasts 10 minutes and allows 5 tries. Codes are never shown on screen on the live server.

The web app

The main channel. It works in any phone browser, can be added to the home screen, opens offline, and keeps voice notes on the phone until the network is back (`web/sw.js`).

Telegram bot (`src/telegram.py`)

- Free, set up with a token from @BotFather (`TELEGRAM_BOT_TOKEN`). At start the app calls `setWebhook` with `PUBLIC_URL/telegram/webhook` and a secret made from the token; every update must carry it in `X-Telegram-Bot-API-Secret-Token`. It also sets the chat's menu button to open `/app`.
- The first time, the bot asks the trader to tap **Share my phone number** (Telegram's `request_contact` button). Only the sender's own contact is accepted, so Telegram has checked the number. The link is kept in `tg_link`, and from then on their messages use that number's book, the same book as the web app.
- Everything else is the WhatsApp bot's code: each update is turned into the message shape `whatsapp.handle` reads, and its replies are sent back through Telegram (`whatsapp.OUT`).
- `/id` tells a team member their chat ID for `TEAM_TELEGRAM` alerts.

WhatsApp bot (`src/whatsapp.py`)

Built and tested, switches on when the Meta keys are set. It is not live: the team could not get WhatsApp Business API access. It uses the Meta Cloud API: `GET /whatsapp/webhook` answers Meta's verify check, `POST /whatsapp/webhook` checks `X-Hub-Signature-256` with `WHATSAPP_APP_SECRET`, and the same brain answers voice notes, photos and text in the trader's language, with buttons for yes and no. Messages outside Meta's 24-hour window need approved templates (`WHATSAPP_TPL_*`, made with `whatsapp_templates.sh`). `eval/test_whatsapp.py` and `eval/test_whatsapp_send.py` test it against a faked Meta. `scripts/check_whatsapp.py` tests the real chain once keys exist.

8. Settings reference

All settings are environment variables, read from `.env` at the repository root. Never put real values in the repository. On the server, use `keys.sh`. Defaults are what the code uses when a setting is missing.

N-ATLaS

Setting	Default	What it does
<code>NATLAS_URL</code>	(none)	N-ATLaS LLM link from Modal, ending in <code>/v1</code> . When set, N-ATLaS is tried first for every record
<code>NATLAS_KEY</code>	(none)	Secret. The same value as in the Modal secret <code>natlas</code> . Sent to both Modal apps
<code>NATLAS_MODEL</code>	<code>natlas</code>	The served model name on the vLLM server
<code>NATLAS_TIMEOUT</code>	20	Seconds N-ATLaS gets for one record before the backups answer
<code>NATLAS_TEMPERATURE</code>	0.1	Model card setting
<code>NATLAS_REPETITION_PENALTY</code>	1.12	Model card setting
<code>NATLAS_ASR_URL</code>	(none)	N-ATLaS speech server link from Modal. When set, voice notes are heard by N-ATLaS only
<code>NATLAS_ASR_PREP</code>	1	Light audio prep on the speech server; 0 turns it off
<code>NATLAS_ASR_MERGE</code>	1	Yorùbá, Hausa and Igbo notes are also heard by the English model and merged by N-ATLaS; 0 turns it off
<code>NATLAS_MERGE_TIMEOUT</code>	8	Seconds to wait for the merge, then the language model's words are used
<code>NATLAS_WATCH</code>	1	Keep N-ATLaS awake in market hours and alert the team; 0 lets it sleep after an hour
<code>NATLAS_WATCH_HOURS</code>	7-20	Market hours, Nigeria time

Other AI models

Setting	Default	What it does
<code>NVIDIA_API_KEY</code>	(none)	Secret. <code>build.nvidia.com</code> key: the Ask chat's main models, photo reading, backups for records. Without it: no photos
<code>LLM_MODELS</code>	<code>nvidia/nemotron-3-super-120b-a12b,google/gemma-4-31b-it</code>	Backup text models after N-ATLaS, in order
<code>VISION_MODELS</code>	<code>google/gemma-4-31b-it,meta/llama-3.2-11b-vision-instruct</code>	Photo models, in order
<code>ASR_MODEL</code>	<code>large-v3-turbo</code>	Whisper model for hearing on your own GPU machine; not used when <code>NATLAS_ASR_URL</code> is set

Setting	Default	What it does
LOCAL_LLM_URL	(none)	Optional own OpenAI-compatible model server (vLLM or NVIDIA NIM), tried last before the rules. Laptop only
LOCAL_LLM_MODEL	Qwen/Qwen2.5-7B-Instruct-AWQ	The model name on that server
LLM_COOLDOWN	120	Seconds a model is skipped after it times out or fails

The Ask chat, live talk and waiting times

Setting	Default	What it does
ASK_BRAIN	llm	rules answers the Ask chat without a model
ASK_MODELS	nvidia/nemotron-3-super-120b-a12b,google/gemma-4-31b-it	Ask chat models. N-ATLaS is added after them, or before them for Yorùbá, Hausa and Igbo
ASK_LLM_SECONDS	45	Time for one Ask message, tools included, before the rules answer
ASK_AI_SECONDS	20	Time one chat or live turn waits for AI before the rules answer
LIST_AI_SECONDS	90	Time N-ATLaS gets to write every line of a pasted list
ANSWER_WAIT_SECONDS	25	After this, longer work continues in the background and the page collects it
VISION_DEADLINE	90	Total seconds for the photo models
LIVE_HEARING	intron	Live talk hearing: intron (streaming) or natlas (one turn at a time)
INTRON_STT_ENGLISH	pcm	Intron's model for English and Pidgin live talk: pcm or en
LIVE_FAST_RECORDS	1	Live talk: a record the rules read completely is answered without waiting for N-ATLaS

Voice (Intron)

Setting	Default	What it does
INTRON_API_KEY	(none)	Secret. Voice replies and live talk; also the hearing comparison tests
TTS_BACKEND	intron when the key is set	off turns voice replies off
INTRON_GENDER	female	female or male

Setting	Default	What it does
<code>INTRON_ACCENT_YORUBA</code> (also <code>_HAUSA</code> , <code>_IGBO</code> , <code>_ENGLISH</code> , <code>_PIDGIN</code>)	built in	Overrides an accent if Intron rejects one
<code>INTRON_ENGLISH_VOICE</code>	<code>pcm</code>	English replies in Intron's Nigerian Pidgin voice; <code>en</code> uses an accented English voice
<code>INTRON_EN_CODE</code>	<code>pcm</code>	Intron code for English and Pidgin in the hearing comparison tests
<code>VOICE_REPLIES</code>	<code>voice</code>	Bots: speak back only after a voice note; <code>always</code> speaks every reply
<code>SPITCH_API_KEY</code>	(none)	Secret. Hearing comparison tests only

Accounts, codes and channels

Setting	Default	What it does
<code>PUBLIC_URL</code>	the request's address	The public <code>https://</code> address: pay links, lender links, the Telegram webhook. <code>setup.sh</code> sets it
<code>SIGNUP_CODE</code>	(none)	<code>required</code> : never sign up without a code (if nothing can send one, only Confirm on Telegram works). Not set on the live server
<code>TEXTBEE_API_KEY</code>	(none)	Secret. Optional SMS codes through TextBee (<code>src/sms.py</code>). Not used by the team; not asked for by <code>keys.sh</code>
<code>TEXTBEE_DEVICE_ID</code>	(none)	Optional: the Android phone that would send the texts
<code>SMS_DAILY_MAX</code>	50	Optional: most code texts a day
<code>TELEGRAM_BOT_TOKEN</code>	(none)	Secret. The Telegram bot; the webhook is set up at start
<code>TELEGRAM_BOT_USERNAME</code>	asked from Telegram	The bot's name for <code>t.me</code> links
<code>WHATSAPP_TOKEN</code>	(none)	Secret. Meta Cloud API token
<code>WHATSAPP_PHONE_ID</code>	(none)	Meta's phone number ID (<code>WHATSAPP_PHONE_NUMBER_ID</code> also works)
<code>WHATSAPP_BOT_NUMBER</code>	(none)	The bot's number, digits only, for the website's WhatsApp buttons
<code>WHATSAPP_APP_SECRET</code>	(none)	Secret. Checks that webhook posts come from Meta; also needed for <code>LOGIN_WORD</code> by message
<code>WHATSAPP_VERIFY_TOKEN</code>	(none)	Secret. Any word; the same word in Meta's webhook settings

Setting	Default	What it does
WHATSAPP_WABA_ID	(none)	WhatsApp Business Account ID, only to create templates by API
WHATSAPP_TPL_CODE , WHATSAPP_TPL_SUMMARY , WHATSAPP_TPL_PAID , WHATSAPP_TPL_ALERT	(none)	Approved template names (for example <code>tradevoice_code</code>) for messages outside Meta's 24-hour window
WHATSAPP_TPL_LANG	en	Template language
WHATSAPP_FLOOD_MAX	30	Messages a minute from one number before the rest are ignored
SHOP_NAME	Chioma Stores	Shop name used when a trader has not given one

Team, privacy and data

Setting	Default	What it does
ADMIN_TOKEN	(none)	Secret. Opens <code>/team?key=...</code> (dashboard and anonymous CSV). Without it <code>/team</code> is closed
TEAM_PHONES	(none)	Team numbers, comma between: marked "Team" and left out of the NAIC numbers
TEAM_WHATSAPP	(none)	Team numbers for alerts by WhatsApp (also left out of the numbers)
TEAM_TELEGRAM	(none)	Team Telegram chat IDs for alerts
PRIVACY_CONTACT	(none)	A team email shown on <code>/privacy</code> (never a personal phone)
DELETE_DAYS	90	Days a deleted account is kept before it is erased
TRAIN_DIR	training next to the books	Where opt-in voice notes, photos and chats are kept
TRAIN_MIN_FREE_GB	3	Below this free disk space, no new audio or photos are kept (the books always save)

Payments

Setting	Default	What it does
PAYSTACK_SECRET_KEY	(none)	Secret. Pay links open a Paystack checkout; the signed webhook settles the debt. Use <code>sk_test_</code> keys until the business is verified
PAYSTACK_EMAIL	(none)	The email Paystack puts on each payment

Backups

Setting	Default	What it does
BACKUP_SUPABASE_URL	(none)	Supabase project link for the off-server copy (used on the live server)
BACKUP_SUPABASE_KEY	(none)	Secret. The project's secret key
BACKUP_SUPABASE_BUCKET	backups	A private bucket
BACKUP_UPLOAD_URL	(none)	Treat as secret. Or an Azure Blob container SAS URL or an Oracle pre-authenticated request URL
BACKUP_COPY_DIR	(none)	Or a folder (for example a mounted drive) that also gets each backup

Local tests only

Setting	Default	What it does
AUTH_DEMO	(none)	1 shows codes on screen. Only on your own computer with made-up numbers. Ignored when TV_PUBLIC=1
AUTH_REQUIRED	1	0 means no login and one shared book (the old demo)
ACCOUNTS_DB , BOOKS_DIR , DB_PATH , BACKUP_DIR	accounts.db , books , tradevoice.db , backups	Where data is kept. On the server they come from /etc/tradevoice/paths.env
PORT	8000	The app's port
TV_PUBLIC	(none)	1 on the live server (set by setup.sh): demo codes can never switch on

setup.sh and keys.sh turn these into comments in the server's .env : ACCOUNTS_DB , BOOKS_DIR , DB_PATH , BACKUP_DIR , PORT , TV_PUBLIC , AUTH_DEMO , AUTH_REQUIRED , AUTH_STRICT , LOCAL_LLM_URL , LOCAL_VISION_URL , ASR_ENGINE .

9. API reference

All paths are on the same server. JSON in and out unless noted. "Login" means the tv_auth session cookie, set by sign-up and log in (HttpOnly, SameSite=Lax, Secure over HTTPS; 90 days with "Keep me logged in"). Voice and photo uploads need the form field consent=yes .

Accounts (no login needed)

Method and path	What it does
POST /api/auth/v2/code/start	{phone, purpose} with purpose signup, login, reset or phone . Sends a code (section 7). Returns login_id, sent, channel, nocode, word, bot, tg

Method and path	What it does
POST /api/auth/v2/code/poll	{login_id}: ok once the number was confirmed by message (WhatsApp LOGIN WORD or Telegram)
POST /api/auth/v2/code/check	{login_id, code}: checks the code
POST /api/auth/v2/signup	{login_id, pw, name, biz, type, mk, lang}: makes the account and logs in
POST /api/auth/v2/login	{phone, pw, keep}: logs in
POST /api/auth/v2/login_code	{login_id, keep}: logs in after a checked code
POST /api/auth/v2/reset	{login_id, pw}: new password; every other device is logged out
POST /api/auth/v2/exists	{phone}: whether the number has an account
POST /api/auth/v2/logout	Ends this session

The trader's account and book (login)

Method and path	What it does
GET /api/v2/me	Profile, devices, deletion date, training answer
POST /api/v2/profile	Name, business, type, market, address, language, notifications, photo
POST /api/v2/password	{current, new}; other devices are logged out
POST /api/v2/devices/logout	{id} or {id: "all"}
POST /api/v2/email	Stores an email (not verified yet)
POST /api/v2/phone	{login_id}: moves the account and book to a new, checked number
POST /api/v2/delete	{biz, pw}: closes the account now, erased after DELETE_DAYS
POST /api/v2/restore	Keeps an account that was going to be erased
POST /api/v2/training	{yes}: the answer to "help make TradeVoice better" (no deletes what was kept)
GET /api/v2/book	Customers who owe, and money in and out for period (today, 7, 30, 365, custom with start, end)
GET /api/v2/customer/{cid}	One customer's history
POST /api/v2/undo_last	Removes the record saved in the last 2 minutes
GET /api/v2/report	Lender report numbers: money in over 30 days, customers, money owed
POST /api/v2/bank_resolve	Account holder's name from Paystack for a payout account

Talk, voice and photos (login)

Method and path	What it does
POST /api/warm	Wakes the N-ATLaS apps; says whether live talk hears with Intron
POST /api/voice	Form: <code>file</code> , <code>session</code> , <code>lang</code> , <code>consent</code> , <code>live</code> . Hears the note, returns the reply and the check card (<code>draft</code>), <code>heard</code> , <code>speak id</code>
POST /api/message	<code>{session, text, lang}</code> : a typed message, or <code>yes / no</code> for the waiting card
POST /api/draft	<code>{session, type, amount, customer, item, quantity, unit, due_date}</code> : "Change" on the card
POST /api/hear	Form: <code>file</code> , <code>lang</code> , <code>consent</code> . Words only; nothing in the book changes
POST /api/say	<code>{session, text, lang}</code> : the reply to words heard in live talk
WS /api/live/hear? lang=...&consent=yes	Live talk: send 16 kHz 16-bit mono audio as binary frames and <code>{"type": "commit"}</code> at the end; receive <code>partial</code> , <code>final</code> or <code>error</code>
GET /api/speak/{id}	A reply's voice note (WAV)
GET /api/speak/{id}/{n}	Piece <code>n</code> of a live reply's voice
POST /api/photo	Form: <code>file</code> , <code>consent</code> . Draft rows from a notebook photo
POST /api/save_rows	<code>{rows}</code> : saves the rows the trader ticked

Ask chat and screen help (login)

Method and path	What it does
POST /api/ask	<code>{session, text, lang, voice}</code> : one chat message; may return <code>{job}</code>
GET /api/job/{id}	Collects a long answer (waits up to 20 s, else <code>{job}</code> again)
POST /api/ask/photo	A photo sent in the chat; may return <code>{job}</code>
POST /api/ask/say	A fresh voice id for an answer's words (replay)
POST /api/ask/reset	"Clear history": the server forgets this chat's memory
GET /api/ask_check?q=...	One question through the Ask chat's model, with which model and tools answered
POST /api/assist, GET /api/explain/{screen}	Questions about the screen; a spoken explanation of it
GET /api/read/{screen}	"Read it to me": the screen as a short spoken summary (<code>/audio</code> for the sound)

Customers and screens (login)

Method and path	What it does
GET, POST /api/customers	List customers; add one (name , phone , notes , credit_limit)
GET, PATCH, DELETE /api/customers/{cid}	One customer and their conversation; edit; delete
POST /api/customers/{cid}/record	A sale or payment for this customer (409 when over the credit limit, unless over_limit_ok)
POST /api/customers/{cid}/say	Text in a customer's conversation: money becomes a draft, anything else a note
POST /api/customers/{cid}/reminder	Drafts a reminder with a pay link; the trader sends it themselves
POST /api/customers/{cid}/statement	Drafts this customer's statement
POST /api/customers/{cid}/confirm_paid	Records a payment the customer said they made
POST /api/customers/{cid}/read, PATCH /api/messages/{mid}	Marks a conversation read; edits a draft message
GET /api/today, /api/debts, /api/insights, /api/profile	Today, who owes and who the trader owes, forecast and best sellers, credit profile and year
GET /api/reminder, /api/repeats; POST /api/repeats/draft	A reminder by name; usual orders due today; one as a draft
GET /api/statement	Business statement (HTML download)
GET /api/export.csv	The whole book as CSV
DELETE /api/entry/{id}; POST /api/wipe	Deletes one record; deletes everything ({"confirm": "DELETE"})
POST /api/demo_data	Fills an empty book with 3 weeks of clearly marked sample records

Lender link, pay links and PIN

Method and path	Login	What it does
POST /api/share, GET /api/shares, DELETE /api/shares/{sid}	yes	Lender link: make (with consent, 1, 7 or 30 days), list (with view counts), stop
GET /lender/{token}	no	The read-only lender page
GET /pay/{token}	no	What the customer owes; pay by Paystack or see the bank details
POST /pay/{token}/paystack, POST /pay/{token}/claimed	no	Start a Paystack checkout; "I have paid"
GET, POST /api/bank	yes	The trader's bank details for pay links

Method and path	Login	What it does
POST, DELETE /api/auth/pin; POST /api/auth/unlock	yes	Set or remove a 4-digit PIN; unlock (returns the X-TV-UnLock value; 5 wrong tries mean a 5-minute wait)

Webhooks

Method and path	Checked by
POST /telegram/webhook	X-Telegram-Bot-API-Secret-Token (made from the bot token)
POST /paystack/webhook	x-paystack-signature (HMAC-SHA512 with PAYSTACK_SECRET_KEY); charge.success settles the debt
GET, POST /whatsapp/webhook	WHATSAPP_VERIFY_TOKEN (verify) and X-Hub-Signature-256 with WHATSAPP_APP_SECRET (messages)

Team dashboard (ADMIN_TOKEN as ?key= or the X-Admin-Key header)

Method and path	What it does
GET /team	The dashboard page
GET /team/api/overview?period=today	Counts for today, 7d or 30d, N-ATLaS share, licence count
GET /team/api/feed	Every action, newest first (codes, never numbers)
GET /team/api/conversations	What traders who said yes said and sent
GET /team/media/{code}/{name}	An opt-in voice note or photo
GET /team/export.csv	The anonymous usage log

Pages and status (no login)

Method and path	What it does
GET /, GET /app, GET /privacy	Website, web app, privacy notice
GET /static/..., GET /sw.js	Files from web/; the service worker
GET /api/status	What the server uses: brain, hearing, voice, photos, keep_awake, whatsapp, telegram, signup_code (no keys, no errors)
GET /api/ui?lang=...	Screen words in a language
GET /api/voice_check?lang=...	Whether voice replies work for a language

The first app's phone-code login (/api/auth/start, /verify, /poll, /guest, /me, /logout, /delete) is still served and used by some tests; the current app does not call it. New work should use /api/auth/v2/ and /api/v2/.

10. Data, privacy and backups

TradeVoice follows the Nigeria Data Protection Act 2023. The privacy notice is at `/privacy`.

Where data lives: on the EC2 server in Ireland (EU), in `/var/lib/tradevoice`, readable only by the app's user.

File	What it holds
<code>books/PHONE.db</code> (one per number)	<code>entries</code> (each record, with the words it was said in), <code>customers</code> , <code>messages</code> (notes, reminder and receipt drafts), <code>reminders</code> , <code>memory</code> (nicknames and usual prices)
<code>accounts.db</code>	<code>users</code> (profile, password hash, PIN hash, bank details, deletion date), <code>logins</code> (code hashes), <code>sessions</code> (token hashes), <code>events</code> (usage log), <code>meta</code> (the log's salt), <code>shares</code> and <code>paylinks</code> , <code>tg_link</code> , <code>training</code>

What is not kept:

- Voice notes and photos are deleted as soon as they are read (`os.remove` in `src/web.py`), unless the trader said yes to helping improve TradeVoice.
- Live talk audio is turned into words and not stored, with the same exception.
- Codes, session tokens and passwords are stored only as hashes.

Usage log (`src/events.py`). One row per action: time, who, channel, kind, language, engine, ok, milliseconds. No names, amounts, words or audio. "Who" is a salted SHA-256 hash of the number (16 characters); the salt is kept only in `accounts.db`. The log is written only by the running server, never by tests. Team phones (`TEAM_PHONES`, `TEAM_WHATSAPP`) are marked "Team" and left out of every count and of the validation report.

Help improve TradeVoice (`src/training.py`). Asked once, separately from sign-up; saying no never limits the app. Only for traders who say yes, their voice notes, photos, live talk audio and chats (with TradeVoice's replies) are kept in `TRAIN_DIR`, one folder per hashed code, never the number. The team sees them on `/team`. Turning it off, or the account being erased, deletes that folder. This folder is not in the backups.

Deleting an account. The account closes at once (`v2.schedule_delete`) and is erased after `DELETE_DAYS` (90) by an hourly job (`v2.purge_deleted`, `accounts.delete_account`): profile, sessions, links, Telegram link, opt-in folder and the book file. Logging in and tapping "Keep my account" before then undoes it. On request from the owner, `erase_account.sh` erases it sooner. Deleting a record or a customer removes it from the book at once.

Backups (`scripts/backup.py`, run hourly by `tradevoice-backup.timer`). Each backup is one file, `tradevoice-YYYYMMDD-HHMMSS.tar.gz`, with consistent SQLite copies of `accounts.db`, `tradevoice.db` and every book. Kept: every backup from the last 48 hours, and the newest of each day for 30 days. Each one is also uploaded to a private Supabase Storage bucket (`BACKUP_SUPABASE_*`), pruned the same way. `restore.sh` puts one back.

Other companies that handle a request: Modal (N-ATLaS models), NVIDIA (Ask chat, photos, backups for records), Intron (live talk hearing, voice replies), Telegram (if the trader uses it), Paystack (pay links), Amazon Web Services (the server) and Supabase (backup copy). Meta only once the WhatsApp bot is switched on. Each gets only what that one request needs.

Security notes. The service runs as a system user with `ProtectSystem=strict`, `ProtectHome`, `PrivateTmp`, `NoNewPrivileges` and write access only to the data folder. Only ports 80 and 443 are open to the internet. The team dashboard needs `ADMIN_TOKEN` on every page, call and file.

11. Tests

Every automated test runs without keys, network or GPU: each suite fakes the services it touches (N-ATLaS, NVIDIA, Intron, Meta, Telegram, TextBee, Paystack) and uses its own temporary databases.

```
python eval/run_all.py          # every suite: N/M per suite and the total
python eval/test_telegram.py    # one suite: prints its checks and "N/M ..."
NODE_PATH=$(npm root -g) node eval/browser_test.cjs  # the app in a real browser
```

On 9 October 2026: 29 suites with 921 checks, all passing, and 75 of 75 browser checks. The browser test needs Node and Playwright with Chromium; it starts its own server with a fresh temporary database and uses made-up names and numbers.

Suite	What it covers
<code>test_auth.py</code> , <code>test_accounts.py</code>	Codes, sessions, sign-up, log in, reset, delete, rate limits
<code>test_natlas.py</code> , <code>test_guards.py</code>	N-ATLaS first with the card settings, backups when it is down; real AI mistakes caught by code
<code>test_hearing.py</code>	Audio prep, unsure words, merging two hearings, what to ask again
<code>test_converse.py</code> , <code>test_chat_smart.py</code> , <code>test_corrections.py</code> , <code>test_list.py</code> , <code>test_clock.py</code>	The conversation: routing, follow-ups, corrections, lists, dates
<code>test_agent.py</code> , <code>test_tools.py</code> , <code>test_ask.py</code> , <code>test_askbook.py</code>	The Ask chat, its tools and number checks
<code>test_live_intron.py</code> , <code>test_intron_tts.py</code>	Live talk relay and voice replies, against a faked Intron
<code>test_whatsapp.py</code> , <code>test_whatsapp_send.py</code> , <code>test_telegram.py</code> , <code>test_sms.py</code>	The bots and the optional SMS codes, against faked services
<code>test_extras.py</code> , <code>test_wholesale.py</code> , <code>test_demo_flow.py</code>	Lender and pay links, PIN, wholesale tools, the demo flow end to end
<code>test_team.py</code> , <code>test_training.py</code> , <code>test_backup.py</code> , <code>test_preflight.py</code> , <code>test_validation_report.py</code> , <code>test_design.py</code>	Dashboard, opt-in data, backups, pilot check, validation report, the front-end build

Accuracy benchmarks (these call real models and need keys):

- `python eval/run_eval.py --cases eval/cases_hard.jsonl --llm natlas`: record accuracy and speed, per language and per trap. Other case files in `eval/`: `cases_1000.jsonl`, `cases_lang.jsonl`, `cases_fresh.jsonl`, `cases_research.jsonl`.

- `python eval/run_eval.py --audio eval/audio --asr natlas`: hearing accuracy on recorded voice notes (`--asr intron` or `spitch` to compare).
- `python eval/run_tools_eval.py --llm`: which tool the model picks, per language.

The results and the commands for each run are in `docs/RESULTS.md`; the method is in `docs/TESTING.md`.

12. Known limits

- **WhatsApp is not live.** The bot is built and tested against a faked Meta, but the team could not get WhatsApp Business API access. It switches on when the Meta keys are set. Until then traders use the web app, and Telegram if they like.
- **N-ATLaS cold start.** After an idle hour the N-ATLaS LLM takes about 3 to 4 minutes to start (the speech server a minute or two). In market hours the server keeps both warm. Outside them, the first message after a quiet hour is answered by the backup models and the rules while N-ATLaS starts.
- **Yorùbá is N-ATLaS's weakest language.** The model card's human evaluation gives Yorùbá 2.69 out of 5, against 4.21 for English. TradeVoice's code checks every record, and fixed reply sentences are used where it matters.
- **Licence cap.** The N-ATLaS terms allow at most 1,000 active end-users in a rolling 30 days. `/team` counts active users and warns at 800. More users, or charging traders, needs an agreement with Awarri and the Ministry.
- **Children's speech is out of scope.** The N-ATLaS speech models list children's speech among their limits; TradeVoice is for adult traders.
- **Wording in Yorùbá, Hausa and Igbo.** The fixed reply sentences and test sentences were written by the team; a native-speaker review is still to do.
- **Numbers are not checked at sign-up.** With no WhatsApp and no SMS sender, a new account is phone number and password only. Telegram users can confirm their number; anyone else who forgets their password needs the team to reset it.
- **One server, one process.** The books are SQLite files with a single writer. This suits a pilot; many thousands of traders would need a different database.